

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Deep Learning

COURSE CODE: MLA0407

COURSE OUTCOME COVERED

CO1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BL3)

Date : 21.07.26

Duration: 1 Hour

QUESTIONS: 1

Total Marks:50

Annexure A

Error Analysis Task

A neural network predicts student scores with an MSE of 0.85 during training and 5.20 during testing. Analyse the possible reasons for this difference and suggest suitable corrective measures.

Code of Conduct:

I Syed Nazma (192425193) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Syed Nazma

ERROR ANALYSIS TASK

Question

A neural network predicts student scores with an MSE of 0.85 during training and 5.20 during testing. Analyse the possible reasons for this difference and suggest suitable corrective measures.

Answer

1. Understanding the Error Difference

The neural network produces a training MSE of **0.85** and a testing MSE of **5.20**.

Mean Squared Error (MSE) is calculated as:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where:

- (y_i) = actual student score
- $(\hat{y}_i)$ = predicted student score
- (n) = number of observations

A lower MSE indicates that the predicted scores are closer to the actual scores.

In this case:

- **Training MSE = 0.85**
- **Testing MSE = 5.20**

The testing error is:

$$\frac{5.20}{0.85} \approx 6.12$$

Therefore, the testing MSE is approximately **6.12 times higher** than the training MSE. This is a significant difference and indicates that the neural network performs much better on the data it has already seen than on new, unseen student data.

The primary issue is **poor generalization**, most likely caused by **overfitting**.

2. Error Identification and Possible Causes

2.1 Overfitting

The most probable cause is **overfitting**.

During training, the neural network adjusts its weights and biases to minimize the error on the training dataset. If the model is trained excessively or is too complex, it may learn:

- Actual relationships in the data
- Noise in the data
- Random variations
- Specific patterns belonging only to training students

As a result, the model produces a very low training MSE of **0.85**. However, when it receives new testing data, it cannot generalize the learned patterns effectively. Consequently, the testing MSE increases to **5.20**.

Example:

Suppose the model learns:

Students who study for 5 hours always score 85.

However, in reality, marks may also depend on:

- Attendance
- Previous knowledge
- Exam difficulty
- Sleep
- Teaching quality
- Subject difficulty

If the model memorizes the training examples instead of learning the general relationship between these factors, it will perform poorly on new students.

2.2 Excessive Model Complexity

The neural network may contain:

- Too many hidden layers
- Too many neurons
- Too many trainable parameters

For example, if the dataset contains only 500 student records but the network has a very large number of parameters, the network may have enough capacity to memorize the training data.

A complex model can achieve:

[\text{Very low training error}]

but:

$$\left[\begin{array}{ccc} \text{High} & \text{testing} & \text{error} \end{array} \right]$$

This represents high variance in the machine learning model.

2.3 Insufficient Training Data

The model may have been trained using a small dataset.

Student performance is influenced by many factors, such as:

- Study hours
- Attendance
- Internal marks
- Previous examination scores
- Assignment performance
- Sleep duration
- Participation in class
- Learning environment

If only a limited number of student records are used for training, the model may not learn all possible patterns.

For example, if the training data contains mainly high-performing students but the testing data contains students with different performance levels, the model may perform poorly on the test set.

2.4 Non-Representative Data Distribution

The training and testing data may not have similar distributions.

For example:

Training Data

Student Type	Percentage
High performers	70%
Average performers	25%
Low performers	5%

Testing Data

Student Type	Percentage
High performers	30%
Average performers	40%

Student Type	Percentage
Low performers	30%

In this situation, the model learns patterns mainly from high-performing students but is tested on a much more diverse population.

This creates a **distribution mismatch** and causes the testing error to increase.

2.5 Data Preprocessing Inconsistency

The training and testing data must undergo the same preprocessing steps.

For example, suppose the training features are normalized using:

$$\left[\begin{array}{l} x' = \frac{x - \mu}{\sigma} \end{array} \right]$$

If the testing data is not normalized using the same training mean (μ) and standard deviation (σ), the neural network receives data in a different scale.

This can produce incorrect predictions and significantly increase the testing MSE.

Correct approach:

1. Calculate scaling parameters using training data.
2. Apply those same parameters to validation data.
3. Apply the same parameters to testing data.

The test data should not be independently fitted during preprocessing.

2.6 Outliers in the Testing Dataset

MSE is highly sensitive to outliers because errors are squared.

Suppose the actual score is 90 and the predicted score is 88:

$$\left[\begin{array}{l} (90-88)^2=4 \end{array} \right]$$

However, if the actual score is 100 and the predicted score is 60:

$$\left[\begin{array}{l} (100-60)^2=1600 \end{array} \right]$$

A few large prediction errors can significantly increase the overall MSE.

Therefore, the testing dataset may contain:

- Incorrect marks
- Data-entry errors
- Unusual student performance
- Extreme values

These outliers may be responsible for the higher testing MSE.

2.7 Training Data Noise

The training data may contain incorrect or noisy information.

For example:

- Incorrect attendance percentage
- Wrong internal marks
- Duplicate student records
- Missing values
- Incorrect study-hour entries

If the neural network learns noise from the training data, it may perform well on the training dataset but fail on real unseen data.

2.8 Improper Train-Test Splitting

The method used to divide the dataset may also be a problem.

If the data is split without randomization, the training and testing datasets may contain different patterns.

For example, if records are arranged according to academic performance:

- First 80% → high-performing students
- Last 20% → low-performing students

Then using the first 80% for training and last 20% for testing creates a biased evaluation.

A proper randomized split should be used.

2.9 Data Leakage

Data leakage occurs when information from the testing dataset accidentally influences the training process.

Examples include:

- Using test labels during training
- Scaling the complete dataset before splitting
- Using future information to predict past performance

- Selecting model features using the entire dataset

Data leakage can produce misleading training performance and unreliable model evaluation.

2.10 Excessive Training Epochs

If the network is trained for too many epochs, it may initially learn useful patterns and later begin memorizing the training data.

The training error may continue decreasing:

[
0.85 $\rightarrow$ 0.60 $\rightarrow$ 0.30
]

However, the validation or testing error may start increasing:

[
3.0 $\rightarrow$ 4.0 $\rightarrow$ 5.20
]

This indicates overtraining.

3. Detailed Corrective Measures

3.1 Apply L1 or L2 Regularization

Regularization adds a penalty to the loss function.

L2 Regularization

[
Loss = MSE + λ $\sum$ w_i^2
]

where:

- (w_i) = model weights
- (λ) = regularization parameter

This discourages extremely large weights and prevents the network from becoming too dependent on particular features.

L1 Regularization

[
Loss = MSE + λ $\sum$ $|w_i|$
]

L1 regularization can also encourage some weights to become zero, effectively reducing the influence of unnecessary features.

3.2 Use Dropout

Dropout randomly disables a percentage of neurons during training.

For example, with a dropout rate of 0.3:

- 30% of neurons are temporarily ignored during each training iteration.
- The remaining neurons learn more independent and robust representations.

This reduces the possibility of the network memorizing the training data.

3.3 Reduce Model Complexity

The network architecture should match the size and complexity of the dataset.

Possible changes include:

- Reducing the number of hidden layers
- Reducing the number of neurons
- Removing unnecessary features
- Reducing the number of trainable parameters

A simpler model can often generalize better than an unnecessarily complex model.

3.4 Use Early Stopping

A validation dataset should be used to monitor performance during training.

For example:

Epoch	Training MSE	Validation MSE
10	2.50	2.80
20	1.50	2.40
30	0.85	3.20
40	0.40	5.20

At epoch 30, the training error continues to decrease, but the validation error increases.

This indicates overfitting.

Therefore, training should be stopped before the validation error starts increasing.

3.5 Increase the Dataset Size

More training examples help the neural network learn general patterns.

The dataset should include students from:

- Different academic backgrounds
- Different performance levels
- Different study patterns
- Different attendance levels
- Different educational environments

A larger and more diverse dataset generally improves generalization.

3.6 Use Proper Data Splitting

A suitable data division may be:

- **70% Training**
- **15% Validation**
- **15% Testing**

or:

- **80% Training**
- **10% Validation**
- **10% Testing**

The training set is used to learn the model parameters.

The validation set is used for:

- Hyperparameter tuning
- Selecting the best architecture
- Detecting overfitting
- Applying early stopping

The testing set is used only for final evaluation.

3.7 Apply Consistent Feature Scaling

Features such as:

- Study hours
- Attendance percentage
- Previous marks
- Assignment scores

may have different ranges.

For example:

- Study hours: 0–15
- Attendance: 0–100
- Previous score: 0–100

Scaling these features helps the neural network learn efficiently.

Standardization can be performed as:

$$z = \frac{x - \mu}{\sigma}$$

The mean and standard deviation should be calculated only from the training dataset and then applied to validation and testing data.

3.8 Detect and Handle Outliers

Outliers should be investigated carefully.

Possible methods include:

- IQR method
- Z-score method
- Box plot analysis

However, valid extreme student scores should not automatically be removed. The data must first be examined to determine whether the value is a genuine observation or an error.

3.9 Use Cross-Validation

K-fold cross-validation can provide a more reliable estimate of model performance.

For example, in 5-fold cross-validation:

1. Divide the dataset into five parts.
2. Train using four parts.
3. Test using the remaining part.
4. Repeat the process five times.
5. Calculate the average MSE.

This helps determine whether the model consistently performs well on unseen data.

3.10 Tune Hyperparameters

The following hyperparameters can be optimized:

- Learning rate
- Number of hidden layers
- Number of neurons
- Batch size

- Number of epochs
- Dropout rate
- Regularization strength

For example, an excessively high learning rate may cause unstable learning, while an unsuitable architecture may cause underfitting or overfitting.

4. Analysis of the Given MSE Values

The difference between the two errors is:

$$[5.20 - 0.85 = 4.35]$$

Therefore, the testing MSE is **4.35 units higher** than the training MSE.

The ratio is:

$$[\frac{5.20}{0.85} \approx 6.12]$$

Thus, the testing MSE is approximately **6.12 times higher** than the training MSE.

The approximate square root of MSE gives RMSE:

Training RMSE

$$[RMSE = \sqrt{0.85} \approx 0.92]$$

Testing RMSE

$$[RMSE = \sqrt{5.20} \approx 2.28]$$

This means that the typical prediction error is approximately:

- **0.92 score units on training data**
- **2.28 score units on testing data**

The testing prediction error is therefore much larger.

5. Interpretation of Model Performance

The model has:

$$[\text{Low Training Error}] + [\text{High Testing Error}]$$

This is the typical pattern of **overfitting**.

The model has learned the training examples very accurately but has not learned general relationships that apply to unseen students.

Therefore, the model has:

- Good training performance
- Poor generalization
- High variance
- Possible sensitivity to noise and outliers

The testing MSE should be considered more important for determining real-world performance because the model will ultimately be used to predict scores for students whose data was not used during training.

6. Recommended Correction Strategy

The most suitable correction strategy is:

Step 1: Check the Data

- Remove duplicate records.
- Correct invalid student scores.
- Handle missing values.
- Identify outliers.
- Check whether the data is representative.

Step 2: Check Data Splitting

- Randomly split the dataset.
- Prevent data leakage.
- Use separate training, validation, and testing datasets.

Step 3: Apply Consistent Preprocessing

- Normalize or standardize numerical features.
- Fit preprocessing only on the training data.
- Apply the same transformation to validation and testing data.

Step 4: Control Model Complexity

- Reduce unnecessary layers and neurons.
- Use L1 or L2 regularization.
- Apply dropout.

Step 5: Monitor Validation Error

- Use early stopping.
- Save the model with the best validation performance.

Step 6: Evaluate the Final Model

- Evaluate only once on the unseen test set.
- Compare training, validation, and testing MSE.
- Use RMSE along with MSE for easier interpretation.

7. Final Conclusion

The training MSE of **0.85** and testing MSE of **5.20** show a large generalization gap. The testing error is approximately **6.12 times higher** than the training error. This strongly indicates that the neural network has learned the training data too specifically and is unable to generalize effectively to unseen student data.

The primary cause is likely **overfitting**, but other factors such as excessive model complexity, insufficient training data, distribution differences, outliers, noisy data, inconsistent preprocessing, improper data splitting, data leakage, and excessive training epochs may also contribute to the error difference.

The problem can be corrected by using:

- A larger and more representative dataset
- Proper train-validation-test splitting
- Consistent data preprocessing
- L1/L2 regularization
- Dropout
- Early stopping
- Reduced model complexity
- Cross-validation
- Hyperparameter tuning
- Proper outlier and data-quality analysis

After applying these corrective measures, the testing MSE should decrease and become closer to the training MSE. This indicates improved generalization and a more reliable neural network for predicting student scores.